CS 296-25 — Initial CLOJURE Homework

1 Introduction and Learning Objectives

1. Ensure that you can start CLOJURE.
2. Use git operations to retrieve your MP.
3. Use `clojure.test` to run tests.
4. Have fun playing with CLOJURE.

2 Getting Started

2.1 Get CLOJURE

Once you have a command line environment running, you need to be sure that CLOJURE is installed. If you are unsure if you have it or not, type

`clj`

and you should see something like this:

```
% clj
Clojure 1.12.0
user=>
```

The numbers may be different. Hit control-D to exit.

2.2 Clone your repository

We have a release repository for code. It is at `https://github.com/mattoxb/cs296-fa26-release.git`. Clone it, but it's probably best if you create your own repository copy assignments to it as they are released.

In the release repository you will find the directory `initial`.

In the `initial` directory you will see the following folders:

- `handout` — this contains ths file `handout.tex`, which is the LaTeX source for the current document.
- `src/initial` — contains the file `core.clj`, which contains the running code for the lab.
- `test/initial` — contains the file `core_test.clj`, which contains the test cases for the lab.

3 Given Code

3.1 The `initial` Namespace

At the beginning of `core.clj`, you will see these lines:

```
1 (ns initial.core)
```

This creates a namespace for your code. We will talk about that more later, but for now you can ignore it.

You need to write four functions. Since this is just practice for getting everything working, we'll have some fun and write politically active arithmetic functions.

- `plus` — This just adds integers together. We wrote this one for you, actually, so it's even easier than it sounds. This is the apathetic citizen who can't be bothered to vote.
- `socialist-plus` — People with more numbers to add should pay their fair share to help out people with fewer numbers. The `socialist-plus` function adds one to sums with fewer than two numbers, and subtracts one for each number after the first 2. See the given code for examples.
- `capitalist-plus` — People with more numbers are better than people with fewer numbers, and mathematical policy should reflect that. After all, it takes numbers to make numbers. The `capitalist-plus` function subtracts one to sums with fewer than two numbers, and adds one for each number after the first 2. See the given code for examples.
- `communist-plus` — Everyone should be equal, no matter how many numbers they have. The state will give you the numbers you need. All sums come out to 10.
- `political-extremist-plus` — Political extremists don't do anything in moderation. They multiply instead of adding. You get to decide which political group would use this function. `:-)`

3.2 Running the Code

If you want to run the code from the command line using `clj`, you can do that as follows:

```
% clj
user=> (require 'initial.core)
nil
user=> (use 'initial.core)
nil
user=> (socialist-plus 10 10 10 10)
38
user=> (plus 10 20 30)
60
```

Note that the `%` and `user=>` strings are prompts. Do not try typing them in.

3.3 Test Cases

The file `test/initial/core_test.clj` contains testing code. It uses `clojure.test`, the testing framework that comes with CLOJURE. You will not write any test cases for this MP, but in the future you will, so do take a look to get an idea how they are written.

To run the test cases, run `clojure -M:test`.

4 Your Work

Your job now is to write the functions and pass the test cases. We are not going to be collecting this; it is for your own enlightenment!